

Sorting Algorithms

1) Bubble Sort

0 1 2 3 4 5 6 7 8
 $A = [-5, 3, 2, 1, -3, -3, 7, 2, 2]$

$j-1 < j \rightarrow j$

$j-1 > j$

swap $\rightarrow [-5, 2, 3, 1, -3, -3, 7, 2, 2]$

$j-1 < j$

$j-1 > j$
swap

do this multiple times
until the array is sorted

Time: $O(n^2)$
Space: $O(1)$

```
def bubble_sort(arr):
```

```
    n = len(arr)
```

```
    flag = True
```

```
    while flag:
```

```
        flag = False
```

```
        for i in range(1, n):
```

→ flip P_1 to $<$ for descending order

```
            if arr[i-1] > arr[i]:
```

```
                flag = True
```

```
                arr[i-1], arr[i] = arr[i], arr[i-1]
```

```
bubble_sort(A)
```

2) Insertion Sort

0 1 2 3 4 5 6 7 8
 $A = [-5, 3, 2, 1, -3, -3, 7, 2, 2]$

$j-1 < j \rightarrow j$

NO swap

swap $\rightarrow [-5, 2, 3, 1, -3, -3, 7, 2, 2]$

swap

$j-1 < j$

NO swap

```
def Insertion_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(1, n):
```

```
        for j in range(i, 0, -1):
```

```
            if arr[j-1] > arr[j]:
```

```
                arr[j-1], arr[j] = arr[j], arr[j-1]
```

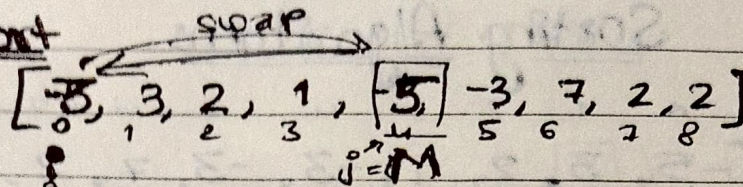
```
            else:
```

```
                break
```

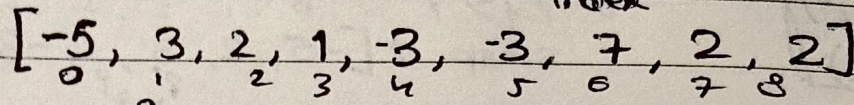
```
Insertion_sort(B)
```

Time: $O(n^2)$
Space: $O(1)$

3) Selection Sort



Time: $O(n^2)$
Space: $O(1)$



def selection_sort(arr):

n = len(arr)

for i in range(n):

min_index = i

for j in range(i+1, n):

if arr[j] < arr[min_index]:

min_index = j

arr[i], arr[min_index] = arr[min_index], arr[i]

selection_sort(c)

4) Merge Sort [divide & conquer algorithm]

D = [-5, 3, 2, 1, -3, -3, 7, 2, 2]

N = len(D)

M = N//2

[-5, 3, 2, 1] [-3, -3, 7, 2, 2]

[-5, 3] [2, 1] [3, -3] [7, 2, 2]

[-5] [3] [2] [1] [-3] [-3] [7] [2, 2]

Base cases

[-5, 3] [1, 2] [-3, -3] [7] [2, 2]

Merge [-5, 3] [1, 2] [-3, -3] [7] [2, 2, 7]

[-5, 1, 2, 3] [-3, -3, 2, 2, 7]

l → l → l → l r → r → r → r → r

[-5, -3, -3, 1, 2, 2, 2, 3, 7]

This is basically a recursive tree

$\log_2 N$

Time: $O(N \log_2 N)$

Space: $O(\log_2 N)$

This turns out to be $O(\log(N))$ when calling

$\log(N)$

def merge-sort(arr):

n = len(arr)

if n == 1:

return arr

m = len(arr) // 2

L = arr[:m]

R = arr[m:]

L = merge-sort(L)

R = merge-sort(R)

~~return arr~~

~~sorted-arr~~

l, r = 0, 0

L-len = len(L)

R-len = len(R)

sorted-arr = [0] * n

p = 0

while l < L-len and r < R-len:

if L[l] < R[r]:

sorted-arr[p] = L[l]

l += 1

else:

sorted-arr[p] = R[r]

r += 1

p += 1

while l < L-len:

sorted-arr[p] = L[l]

l += 1

r += 1

while r < R-len:

sorted-arr[p] = R[r]

r += 1

l += 1

return sorted-arr

5) Quick Sort

E = [-5, 3, 2, 1, -3, -3, 7, 2, 2]

Pivot

elements that are less than pivot ↓ elements that are greater than pivot

[-5, 2, 1, -3, -3, 2] [2]

[3, 7]

we do this recursively

Bad Pivot!!

[-5, 2, 1, -3, -3] + [2] + [3, 7]

[-5, 3] [-3] [2, 1]

[-5] [-3] [1, 2] + [3, 7] + [2]

Since the elements at the end are already in sorted order hence, merging will just basically be the concatenation of the lists.

$[-5, -3] \quad [-3] \quad [1, 2]$

$[-5, -3, -3, 1, 2] \quad [2] \quad [3, 7]$

$[-5, -3, -3, 1, 2, 2] \quad [2] \quad [3, 7]$

Time: $O(N \log_2 N)$
 $O(N^2)$
 Space: $O(\log N)$

[If good pivot]

[absolute worst case when pivot is bad repeatedly]

[$O(N)$ during coding]

def quick-sort(arr):

if len(arr) <= 1:

return arr

p = arr[-1]

L = [x for x in arr[:-1] if x <= p]

R = [x for x in arr[:-1] if x > p]

L = quick-sort(L)

R = quick-sort(R)

return L + [p] + R

quick-sort(E)

6) Counting Sort

$F = [5, 3, 2, 1, 3, 3, 7, 2, 2]$

Iterate $\uparrow$
over array

$K = \text{Maxx} = 7$

$0 \rightarrow 1 \rightarrow 1 \rightarrow 1 \rightarrow 1 \rightarrow 1 \rightarrow 1 \rightarrow 1$

(max value from the given array)

Counts: $\begin{bmatrix} 0 & 0^1 & 2^3 & 2^3 & 0 & 1 & 0 & 1 \\ 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 3 & 3 & 0 & 1 & 0 & 1 \\ 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \end{bmatrix}$

rewrite the original array now

$[1, 2, 2, 1, 3, 3, 7, 2, 2]$

Time: $O(K+N)$

Space: $O(K)$

if K is small, then this algorithm is really good for it

def counting_sort(arr):

$n = \text{len}(arr)$

$\text{maxx} = \max(arr)$

$\text{counts} = [0] * (\text{maxx} + 1)$

for x in arr :

$\text{count}[x] += 1$

$i = 0$

for c in range($\text{maxx} + 1$):

while $\text{counts}[c] > 0$:

$arr[i] = c$

$i += 1$

$\text{counts}[c] -= 1$

counting_sort(F)

Sorting in Practice

Time complexity: $O(N \log N)$ using Sort

$G = [-5, 3, 2, 1, -3, -3, 7, 2, 2]$

$G = \text{sort}()$ # sorts in place

$\rightarrow G$

gets change

$G = [-5, -3, -3, 1, 2, 2, 2, 3, 7]$

Get new sorted array - $O(n)$ space

$H = [-5, 3, 2, 1, -3, -3, 7, 2, 2]$

$\text{sorted_H} = \text{sorted}(H)$

$H, \text{sorted_H}$

$([-5, 3, 2, 1, -3, -3, 7, 2, 2], [-5, -3, -3, 1, 2, 2, 2, 3, 7])$

Sort array of tuples

$I = [(-5, 3), (2, 1), (-3, 3), (7, 2), (2, 2)]$

$\text{sorted_I} = \text{sorted}(I, \text{key} = \text{lambda } t: t[0])$

sorted_I

add
rearrange to 10th
in descending
order

$[(-5, 3), (-3, 3), (2, 1), (2, 2), (7, 2)]$

used when solving Interval Problems